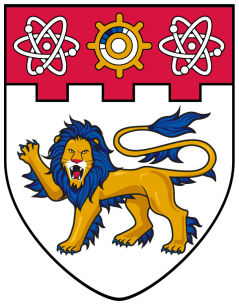


NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

Semester Group Project

by

Name	Matric No.	Work %
Timothy Chang	*U2220136J*	50.0%
Neo Zhi Xuan	*U2222293E*	50.0%

SC4023 Big Data Management

NANYANG TECHNOLOGICAL UNIVERSITY

April 2026

Abstract. We present our highly performant columnar store query engine. It implements **15 distinct optimisations**, spanning five architectural layers. They include encoding (Dictionary, Precomputed aggregates, RLE town compression), Physical layout (Pre-sorted clustering, Per-column binary files, Town partitioning), Indexing (Zone maps, Bitmap indices, Binary search), Scan-path rewriting (Result reuse, Late materialisation, Predicate reordering, Integer gating), and I/O management (Chunked streaming, Memory-mapped I/O). All optimisations are toggleable by boolean flags. This enables us to build a combinatorial evaluation suite testing across 40 permutations of optimisations, each with correctness verification. Physical town partitioning together with Result reuse achieves a **3,389× query speedup** at only 660 KB memory. Memory-mapped I/O with Result Reuse has the fastest end to end time of **71.9 ms**, 67.3× faster than baseline. Negative results are instructive as well. Bitmap indexing (0.15×) and Predicate reordering (0.06×) show that remove cheap predicates via an expensive process greatly affects performance, especially when data is already well filtered upstream.

1 Data Storage

1.1 System Architecture

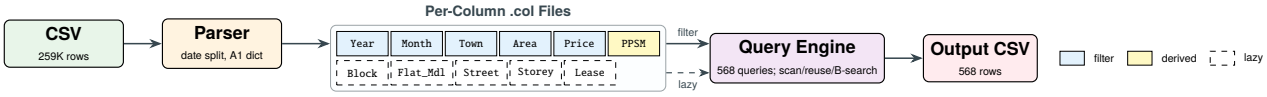


Figure 1: Pipeline: CSV → parser → per-column binary files → engine → output CSV.

Our parser detects column positions from the CSV header and streams rows into parallel `std::vector` columns. `YYYY-MM` date strings are split into `Year` (`uint16`) and `Month` (`uint8`). Invalid rows (malformed dates, empty fields, etc) are skipped.

1.2 Baseline Column Store

The `ColumnStore` struct holds one `std::vector` per column. Numeric columns use fixed-width types as seen in Table 1. All columns are populated in one pass through the CSV. For the baseline, loads of all 10 columns into memory produces a total footprint of 71.4 MB.

1.3 Per-Column Physical Layout

Column	Type	Encoding	B/row	Derived?	Rationale
Year	uint16	raw	2	from `YYYY-MM`	range predicate
Month	uint8	raw	1	from `YYYY-MM`	range predicate
Town	uint8	dict (A1)	1	no	26 towns ≤ 5 bits
Floor_Area	uint16	raw	2	no	$\geq y$ predicate
Resale_Price	uint32	raw	4	no	aggregate target
PricePerSqm (A4)	float32	raw	4	yes	removes per-row division
Block, Flat_Model, Lease_Commence	...	late-materialised (§2.5)			output only

Table 1: Physical layout per column.

1.4 Encoding and Layout Decisions

Dictionary encoding (A1). Our `DictionaryEncoder` is a bidirectional `unordered_map/vector` pair. It maps each of the unique 26 towns to a `uint16_t` ID. During query, our engine would first resolve the target town list to IDs once outside the scan and then use integer equality in the inner loop. This brought about a 32% query speedup (2,299→1,573 ms) at the cost of +2.9% memory and 16.7% slower load time. The one load time cost is a penalty amortised across all the queries.

Column splitting of `Month`. We split the date string into two integer columns at parse time. This eliminates repeated string parsings and allow for direct `uint16` range predicates. With just the Year predicate integer comparision, we eliminated ~90% of rows.

Precomputed PricePerSqm. The storing of `price / area` as `float32` will reduce the per-row floating point division we had to do for entries that survived the filter. Only ~0.2% of row iterations reach this division, the ~543 survivors per query. This is why the measured speed up is only <0.1%.

Pre-sorted clustering. A `stable_sort` by the key (Town, Year, Month) will physically reorder all columns once at ingestion. A linear pass then records each town partition's `[begin, end)` boundaries at a $O(1)$ lookup cost at query time. This is the foundation prerequisite for binary search (§2.4) and zone map chunks (§1.5).

Separate binary column files. Every column is written to its own binary file during the CSV to columnar conversion. During query time, only the five filtered columns are loaded. Output columns are lazy loaded for surviving row per (x, y) pairs. This drops memory usage from 71.4 MB to 2.7 MB, a 96.2% reduction. Load time also falls from 2,536 ms to 68 ms. The value of this optimisation is in memory and load-time reduction. It does not have any improvements in query time and in fact is 2.0% *slower* because the binary I/O path still performs a full scan.

1.5 Zone Maps

Every numeric filter column is divided into fixed size chunks of 1,024 rows, where its `(min, max)` entry is stored. There hence will be 254 chunks in total, with a size of <8 KB. With §1.4, a skip rate of 96.1% is achieved across all 568 queries (138.6K of 144.3K chunks skipped).

1.6 Bitmap Index on Town

We build one `std::vector<bool>` for each distinct town at load time (26 bit-vectors × 259K bits ≈ 842 KB). Before querying, bit vectors for the specific towns are OR combined into a `uint8_t`-per-row mask (~259 KB). This replaces the per-row town predicate with a single array lookup. The mask uses one byte per row for cache line friendly sequential access. As explored in §3.5, this optimisation hurts performance. Eliminating all 12.55M Town comparisons yielded just 0.15× throughput. Its overhead outweighted the cost of original comparisions as the Year predicate already filters out ~90% of rows before Town is reached.

1.7 I/O Layer: Chunked and Memory-Mapped

Chunked I/O. Our system streams filtered columns from §1.4 in bounded size chunks under a configurable memory budget. The outer loop iterates over the I/O chunks while the inner loop runs all 568 queries per chunk. It exploits the associativity of `min` to merge per-chunk winners.

Memory-mapped I/O. This feauture replaces `ifstream/fread` with POSIX `mmap(2)`. It means each `.col` file is mapped into virtual address space with a single syscall, then `memcpy`'d into the existing column vectors. Load times drop from 68 ms (A9 `ifstream`) to 31 ms , a 55% reduction. For lazy materialisation, `loadStringAtMmap` replaces `std::ifstream` construction with raw POSIX calls. Composed with §2.3, it achieves the fastest total time of **71.9 ms** (67.3× end-to-end).

1.8 Physical Partitioning by Town

Columnar binary files are split into one subdirectory per town. This is how we implemented partition pruning. During a query, `loadColumnFilesPartitioned` opens only the subdirectories for the target town and concatenates their columns into the in-memory `ColumnStore`:

Listing 1: Partition loading (simplified).

```
void loadColumnFilesPartitioned(const string& base_dir,
       const vector<string>& target_towns, ColumnStore& db) {
    for (const auto& town : target_towns) {
        string dir = base_dir + "/" + town;
        // append filter + materialisation columns from dir
        appendNumericCol(db.col_month_year, dir+"/month_year.col");
        appendNumericCol(db.col_floor_area, dir+"/floor_area.col");
        // ... remaining columns ...
    }
    // rebuild zone maps / bitmap indices over concatenated data
}
```

For 3 target towns, only 42,935 of 259,237 rows are loaded, a 83.4% reduction. If a row is in memory, it implicitly means that it must belong to a target town. No other town comparisions are then needed. Memory footprint also drops to **660 KB**, which is 99.1% less than baseline.

1.9 RLE on Sorted Towns

We implement run-length encoding on sorted town columns by compressing `town_encoded.col` into (value, start, length) triples:

$$\texttt{run\_value}[k],\ \texttt{run\_start}[k],\ \texttt{run\_length}[k].$$

The query engine resolves requested towns to run IDs and scans only the selected intervals, skipping non-matching runs entirely instead of checking town equality for every row. Without pre-sorting (A5 alone), the unsorted column produces 2,570 runs; the engine still skips 122.86M rows via run boundaries, reducing rows scanned post-RLE to 24.39M and achieving a 10.5× query speedup. With pre-sorting (A2+A5), each town forms a single contiguous run (26 runs total), and town comparisons drop to zero. The strongest composition is B3+A2+A5: binary search narrows each town partition to the target month range, and RLE eliminates all remaining town equality checks, yielding a 180× query speedup (13.7 ms query time).

1.10 Exception Handling

In empty result sets, we return a "No result" row in our output. Ties in minimum price-per-sqm are resolved by first encountered order.

1.11 Space–Time Trade-off Summary

Decision	ΔMem / ΔLoad	ΔQuery	Verdict
Dict-encode Town (A1)	+2.9% mem, +16.7% load	−32% query	keep
Precompute PricePerSqm (A4)	+2.8% mem	±0% query	keep (negligible)
Pre-sort (Town,Y,M) (A2)	+16.6% load	enables B3	keep (foundation)
Columnar files (A9)	−96.2% mem, −97.3% load	+2.0% query	keep (I/O win)
Zone maps (B1)	$+\epsilon$ mem	−25% query	keep
Bitmap index Town (B2)	+1.1% mem	−582% query	reject
mmap I/O (D2)	~ mem	−55% load	keep (I/O)
Town partitioning (E1)	−99.1% mem	−90.9% query	keep
RLE Town column (A5)	$+\epsilon$ mem	−90.5% query	keep (with A2)

Table 2: Storage decisions as explicit trade-offs. All deltas vs. baseline.

2 Data Processing

2.1 Baseline Scan

The baseline scans all 259,237 rows for each of 568 (x, y) pairs. Predicates are evaluated in the order Year → Month → Town → Area; Year is checked first because it eliminates ~90% of rows with a single `uint16` equality before the more expensive Town iteration runs.

Listing 2: Baseline scan loop in `runQuery` (simplified).

```
for (size_t i = 0; i < N; ++i) {
    if (db.col_month_year[i] != target_year) continue; // ~90% eliminated
    if (db.col_month_month[i] < start_month ||
        db.col_month_month[i] > end_month) continue;
    // Town: linear scan over 7-element target list
    bool match = false;
    for (const auto& t : towns)
        if (db.col_town[i] == t) { match = true; break; }
    if (!match) continue;
    if (db.col_floor_area[i] < y) continue;
    double ppsm = (double)db.col_resale_price[i] / db.col_floor_area[i];
    if (ppsm < min_ppsm) { min_ppsm = ppsm; best_i = i; }
}
```

Across the full grid this produces 147.25M row iterations and 12.55M town comparisons, averaging 4,835.6 ms total (2,536.1 ms load + 2,299.5 ms query).

2.2 Composable Scan Architecture

All 15 optimisations are controlled by independent boolean flags in the `ColumnStore` struct (toggled via `OptConfig` in the evaluation suite). The query engine has three paths: (1) a *reuse path* (C1+C2) that short-circuits via the cumulative table (§2.3), (2) a *unified scan loop* where each flag controls one orthogonal decision—A1 selects int vs. string comparison, B1 wraps the loop in chunk-level pruning, B2 replaces the town predicate with a bitmap mask, A5 replaces per-row town checks with run-level skipping, C4 reorders predicates, C6 gates float computation, A4 selects the PPSM source, and C3 defers the price column to a Phase 2 survivors pass, and (3) a *partition-pruned path* (E1) that loads only target-town data before entering either path (1) or (2). D2 (mmap) is an I/O-layer substitution under A9, transparent to the scan logic. This composability allows the evaluation suite to test ~40 configurations and isolate each contribution.

2.3 Result Reuse (C1+C2)

The 568 queries overlap: for fixed y, the month window for $x{=}k$ is a superset of $x{=}k{-}1$; for fixed x, every record with `area` $\geq y$ also satisfies `area` $\geq y{-}1$.

Three-step cumulative table build. Step 1 (single $O(N)$ scan): For each row passing year and town filters, compute *offset* = `month` − *start* + 1 (range 1–8) and *bucket* = min(`area`, 150). Records with `area` > 150 are *clamped* to bucket 150, not dropped—this is correct because every valid y threshold is ≤ 150, so a record with area 160 satisfies every y and must contribute to all buckets it qualifies for; the clamped bucket propagates into all lower buckets during the Y-sweep. **Step 2 (X-sweep):** For each area bucket, sweep offsets 1 → 8, propagating a running minimum. **Step 3 (Y-sweep):** For each offset, sweep area 149 → 80, propagating the minimum downward. After Step 3, `cum_table[x][y]` holds the answer for each (x, y) pair, and the query phase becomes a single array lookup. The full C++ implementation is given in Figure 7 (Appendix).

Results. C1+C2 alone reduces query time from 2,299.5 ms to 4.6 ms (499× query speedup), with rows scanned dropping from 147.25M to 259.2K (the single build scan). When composed with dictionary encoding (A1+C1+C2), query time drops to 3.1 ms (749×).

2.4 Pre-sort + Binary Search (A2+B3)

With data pre-sorted by (Town, Year, Month), the query engine runs custom `lowerBound/upperBound` searches within each town partition using a linearised month key (`year` × 12 + `month` − 1):

Listing 3: Binary search on linearised month key.

```
inline uint32_t monthKey(uint16_t year, uint8_t month) {
    return (uint32_t)year * 12u + (uint32_t)(month - 1u);
}
size_t lowerBoundMonthKey(const ColumnStore& db, size_t l, size_t r,
                          uint32_t key) {
    while (l < r) {
        size_t mid = l + (r - l) / 2;
        if (monthKeyAt(db, mid) < key) l = mid + 1; else r = mid;
    }
    return l;
}
```

Rows scanned drop from 147.25M to 751.7K (99.5% reduction), town comparisons fall to zero, and query time drops to 11.9 ms (193× query speedup). This is an independent scan-elimination path from C1+C2.

2.5 Late Materialisation (C3)

Phase 1 applies only the four predicate columns, appending survivor indices to a vector. Phase 2 iterates survivors to fetch Resale_Price, compute PricePerSqm, and track the minimum. Measured result: 0.7% *slowdown* (2,315 ms vs. 2,299 ms), essentially within measurement noise, because the overhead of maintaining the survivor vector roughly cancels the cache savings from deferring a single 4-byte column.

2.6 Other Scan Optimisations

Zone-map chunk skipping (B1). Before scanning a 1,024-row chunk, the engine tests its min/max against Year, Month, and Floor_Area predicates; failing chunks are skipped entirely (96.1% skip rate, 25% standalone query speedup; combined with A1: 58%).

Bitmap-driven row filtering (B2). A precomputed `uint8_t` mask replaces the per-row town loop with a single byte lookup. Town comparisons drop to zero, but query time regressed to 0.15× (15,670 ms vs. 2,299 ms baseline): the byte-load-plus-branch on every row is more expensive than the original integer comparison that runs only on the ∼10% of rows surviving the Year filter. This confirms that the baseline Year→Month→Town order is already near-optimal.

Predicate reordering (C4). Reordering to Town → Year → Month → Area caused a 0.06× regression: town comparisons surged from 12.55M to 417.51M because every row now pays the expensive Town list iteration before the cheap Year check can eliminate it. The lesson: optimal order must minimise *cost* × *pass_rate*, not maximise selectivity alone.

Integer early-exit gate (C6). Tests `price` × `cur_min_area` ≥ `cur_min_price` × `area` in `uint64` before computing float PricePerSqm. Measured: within noise—too few survivors (∼543/query) for the eliminated float work to register.

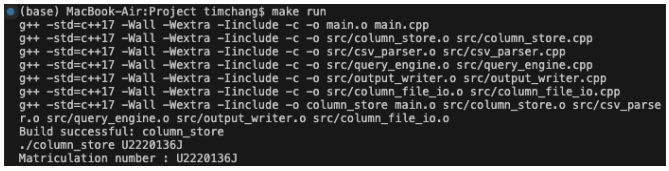
2.7 Correctness Argument

Each optimisation preserves output equivalence: **A1** dictionary encoding is a bijection. **A2** sorting is a permutation. **A4** precomputed PPSM is identical to runtime division. **A9** binary files reproduce identical column vectors. **B1** zone maps skip a chunk only when its (min, max) proves no row can match. **B2** bitmap mask is the OR of exact per-town bit-vectors; the mask lookup is equivalent to the membership test it replaces. **B3** binary search locates exact boundaries in sorted data. **C1+C2** result reuse is exact because min is associative over disjoint month ranges and monotone over area thresholds; clamping area > 150 to bucket 150 is correct because all valid $y \leq 150$. **C3** defers column access without altering the predicate set. **C4** changes evaluation order, not the predicates. **C6** is a sufficient condition for "cannot beat current minimum." **D1** processes the same rows in bounded batches; the cumulative table is merged across chunks via the associativity of min. **D2** `memcpy` from mapped region produces byte-identical vectors as `fread`. **A5** RLE run boundaries mark contiguous blocks of identical town values; skipping a non-matching run is equivalent to skipping each row individually. **E1** partition loading concatenates disjoint town subsets; the union equals the set of rows matching the town predicate. All ∼46 configurations produced byte-identical output, verified by the evaluation suite's per-result hash comparison (§3.4).

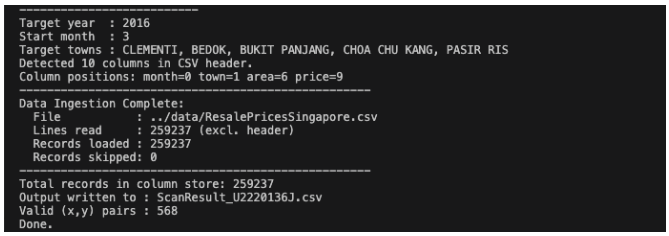
3 Experiment Result

Our query parameters are derived from matric number `U2220136J`: target year 2016, start month 3 (March), threshold ≤ 4725, towns {CLEMENTI, BEDOK, BUKIT PANJANG, CHOA CHU KANG, PASIR RIS}, with $x \in [1, 8]$ and $y \in [80, 150]$ yielding 568 (x, y) query pairs.

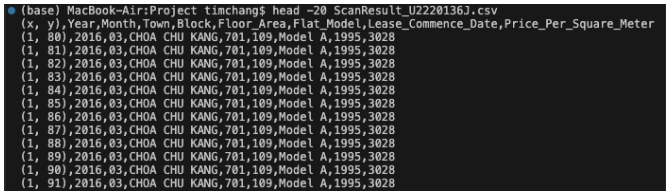
3.1 Compulsory Artefacts



(a) Compilation.



(b) Execution (259,237 rows, 568 queries).



(c) `ScanResult_U2220136J.csv`.

Figure 2: End-to-end execution: build (a), ingestion + query (b), output CSV (c).

3.2 Excel Cross-Validation

We wrote an independent Office Script to verify engine outputs. The script reads the raw CSV dataset, applies the same four predicates (Year = 2016, Month $\in$ [start, start+$x-1$], Town $\in$ {CLEMENTI, BEDOK, BUKIT PANJANG, CHOA CHU KANG, PASIR RIS}, Floor_Area $\geq y$), and computes min(`Resale_Price`/`Floor_Area`) rounded to the nearest integer—identical logic to the engine's `runQuery`. Table 3 confirms exact agreement across six (x, y) pairs spanning different selectivities; the Appendix (Figure 5) shows the script environment and output alongside the engine's CSV.

x	y	Engine PPSM	Excel PPSM	Match
1	80	3028	3028	✓
3	90	2878	2878	✓
4	100	2788	2788	✓
6	120	2878	2878	✓
7	130	2939	2939	✓
8	150	3387	3387	✓

Table 3: Excel cross-validation for selected (x, y) pairs.

3.3 Benchmark Table

Each configuration was benchmarked; memory is estimated from column vector capacities plus dictionary/zone-map overhead. Table 4 shows a cumulative build-up revealing three independent optimisation paths: result reuse (C1+C2), pre-sort + binary search (A2+B3), and physical town partitioning (E1).

Configuration	Load (ms)	Query (ms)	Total (ms)	Q.Speedup	Rows Scan.	Memory
Baseline	2,536.1	2,299.5	4,835.6	1.0×	147.25M	71.4 MB
+ A1 Dict Encoding	2,958.5	1,572.7	4,531.2	1.5×	147.25M	73.4 MB
+ C1+C2 Reuse (alone)	2,518.7	4.6	2,523.3	499×	259.2K	71.4 MB
+ A1+C1+C2 Dict+Reuse	2,920.5	3.1	2,923.6	749×	259.2K	73.4 MB
+ A9+C1C2 Columnar+Reuse	78.5	46.6	125.1	49.4×	259.2K	2.7 MB
+ A9+All Everything	25.0	45.7	70.7	50.3×	259.2K	5.3 MB
Memory-mapped I/O path (D2):						
D2+C1C2 mmap+Reuse	33.5	38.4	**71.9**	59.9×	259.2K	5.3 MB
D2+All mmap+Everything	41.0	45.5	86.5	50.5×	259.2K	6.1 MB
Alternative scan-elimination path (A2+B3):						
A2+B3 Pre-sort+BSearch	3,293.5	11.9	3,305.4	193×	751.7K	71.0 MB
A9+A2+B3 Col+Sort+BSearch	115.3	69.3	184.5	33.2×	751.7K	2.7 MB
RLE town skipping path (A5):						
A5: RLE Town	2,526.0	234.7	2,760.7	10.5×	24.4M	71.5 MB
A2+A5 Presort+RLE	2,981.1	224.7	3,205.8	10.9×	24.4M	71.0 MB
B3+A2+A5 BSearch+Presort+RLE	2,979.8	13.7	2,993.5	180×	751.7K	71.0 MB
A9+A2+A5+B3 Col+Sort+RLE+BSearch	104.5	55.8	160.3	44.0×	751.7K	2.7 MB
Partition-pruned path (E1):						
E1: Town Partition	72.0	208.4	280.4	11.0×	24.4M	660 KB
E1+B1 Partition+ZoneMaps	74.1	168.0	242.1	13.7×	24.4M	1.2 MB
E1+C1C2 Partition+Reuse	72.4	0.7	73.1	**3,389**×	42.9K	660 KB
E1+All Partition+Everything	73.5	0.7	74.2	3,403×	42.9K	1.2 MB

Table 4: Benchmark across key configurations. Q.Speedup = baseline query time / config query time.

Figure 3: Speedup vs. baseline (= 1.0×) across key configurations.

3.4 Correctness Invariant

The evaluation suite (`eval_suite.cpp`) compares each configuration's 568-entry output vector against the baseline element-by-element, checking that the (x, y) pair, all output fields, and the computed PricePerSqm are identical. All ~46 configurations report PASS.

3.5 Negative and Neutral Results

The professor explicitly requested trade-off justification (clarification Q1). Table 5 isolates five optimisations that underdelivered. C4 and B2 are the most instructive: both are textbook-recommended techniques that regressed dramatically because they violated cost×pass_rate ordering.

Configuration	Query (ms)	Δ vs Baseline	Verdict
C4: Predicate Reorder	37,584.1	0.06×	Rejected
B2: Bitmap Town Index	15,669.9	0.15×	Rejected
C3: Late Materialise	2,315.5	0.99×	Within noise
A4: Precompute PPSM	2,295.9	1.00×	Within noise
C6: Int Multiply Gate	2,298.4	1.00×	Within noise

Table 5: Negative and neutral results (individual isolation).

B2 is instructive alongside C4: both attempt to eliminate town comparisons, yet B2 *regressed* because the per-row byte-load adds branch overhead on all 147M row-visits, while A2+B3 avoids visiting non-target rows entirely. C3 produced a neutral result because deferring a 4-byte column on a narrow schema yields cache savings that roughly cancel against the overhead of maintaining a survivor index vector.

3.6 Sensitivity Analysis: D1 Memory Budget

Table 6 varies the D1 memory budget across a 50× range. All D1 configurations use A9 columnar files with A1, B1, and result reuse built into the batch runner.

Budget	I/O Chunks	Query (ms)	Total (ms)	Q.Speedup
50 MB	1	372.6	405.2	6.17×
10 MB	1	371.6	395.9	6.19×
5 MB	2	373.9	397.7	6.15×
2 MB	3	375.1	398.7	6.13×
1 MB	6	401.6	425.4	5.73×

Table 6: D1 chunked I/O sensitivity. Total bytes read = 4.7 MB at all budgets.

Performance is stable from 50 MB down to 2 MB, degrading 7% at the 1 MB extreme where 6 I/O chunks are required. Byte-identical correctness is maintained at every budget.

3.7 Environment

All benchmarks were run on an Apple M-series MacBook Air, compiled with `clang++ -O2 -std=c++17`. The dataset contains 259,237 rows across 10 columns; all 568 (x, y) query pairs were evaluated per configuration.

4 Conclusion

The largest query-time win was result reuse (C1+C2): a single $O(N)$ build pass replaced $568 \times O(N)$ scans with $O(1)$ lookups (499× query speedup; 749× with A1). Physical town partitioning (E1) reduced the build-pass input from 259.2K to 42.9K rows, yielding 3,389× query speedup and 660 KB memory—the most memory-efficient configuration. The fastest end-to-end configuration was D2+C1C2 (mmap + reuse) at 71.9 ms total (67.3×), where mmap's 55% load-time reduction composed with reuse's scan elimination. The most instructive failures were C4 predicate reordering (0.06×) and B2 bitmap indexing (0.15×): C4 demonstrated that optimal predicate order must minimise cost × pass_rate, while B2 showed that eliminating a cheap predicate via a more expensive mechanism can regress performance when upstream predicates already prune effectively. Across all 15 optimisations and ~40 tested configurations, every result was byte-identical to the baseline—correctness was never sacrificed for speed. If revisiting this project, we would explore a cost-based predicate optimiser that dynamically selects evaluation order.

Appendix: Figures

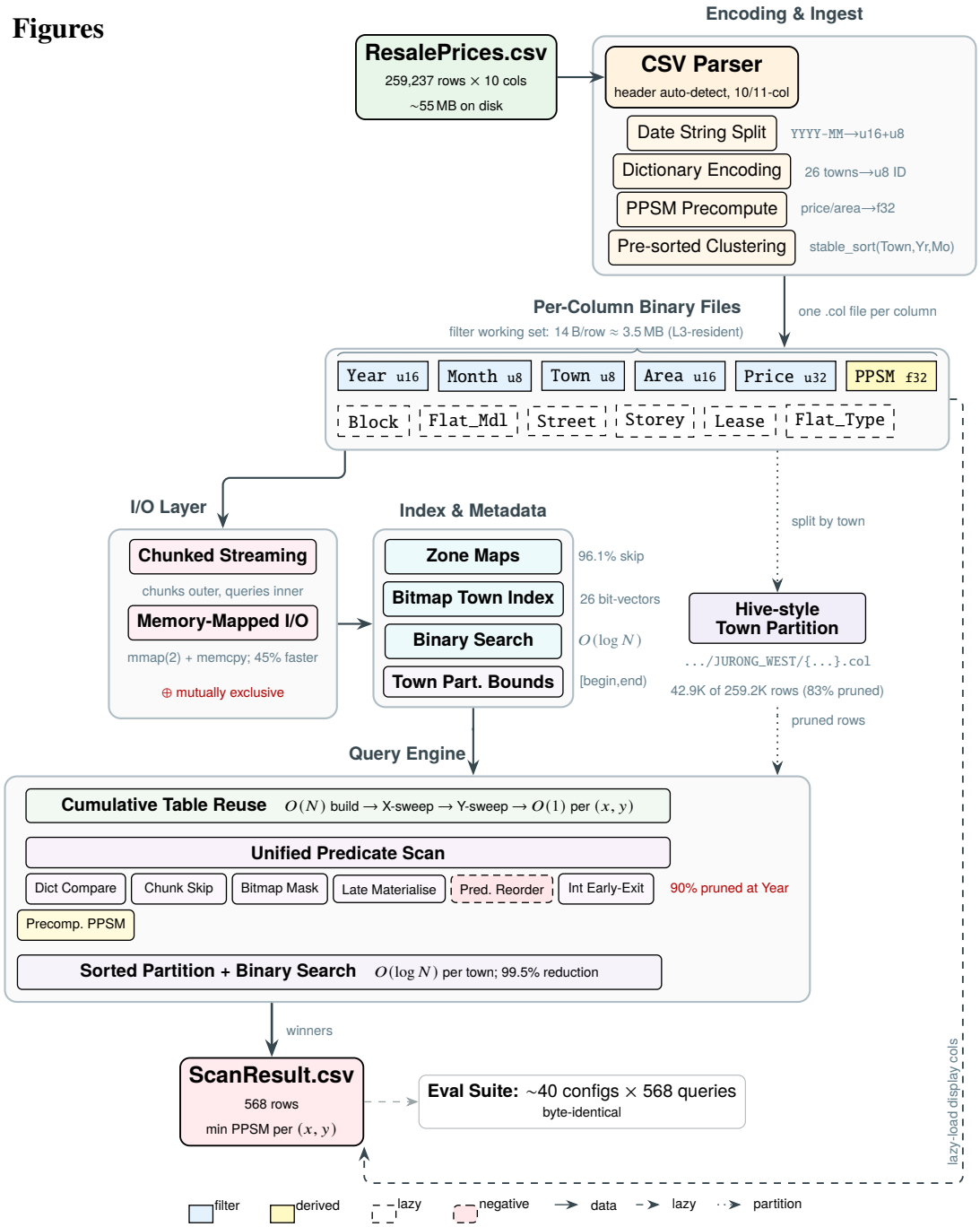


Figure 4: Data Flow and Processing Architecture

SC4023 Excel Cross-Validation — Matric U2220136J								
Year=2016 StartMonth=3	Towns: CLEMENTI, BEDOK, BUKIT PANJANG, CHOA CHU KANG, PASIR RIS							
(x, y)	Excel_PPSM	Valid?	Year	Month	Town	Block	Floor_Area	Flat_Model	Lease_Commence_Date
(1, 80)	3028	YES	2016	3	CHOA CHU KANG	701	109	Model A	1995
(3, 90)	2878	YES	2016	5	CHOA CHU KANG	464	123	Premium Apartment	2000
(4, 100)	2788	YES	2016	6	BUKIT PANJANG	213	104	Model A	1988
(6, 120)	2878	YES	2016	5	CHOA CHU KANG	464	123	Premium Apartment	2000
(7, 130)	2939	YES	2016	7	CHOA CHU KANG	540	132	Model A	1994
(8, 150)	3387	YES	2016	6	PASIR RIS	219	150	Apartment	1993

(a) Office Script editor and Validation sheet output.

ScanResult_U2220136J.csv | ScanResult_U2220136J validation.csv | Makefile | README.md

Project > ScanResult_U2220136J validation.csv

```
7  (x, y),Year,Month,Town,Block,Floor_Area,Flat_Model,Lease_Commence_Date,Price_Per_Square_Meter
6  (1, 80),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
5  (3, 90),2016,05,CHOA CHU KANG,464,123,Premium Apartment,2000,2878
4  (4, 100),2016,06,BUKIT PANJANG,213,104,Model A,1988,2788
3  (6, 120),2016,05,CHOA CHU KANG,464,123,Premium Apartment,2000,2878
2  (7, 130),2016,07,CHOA CHU KANG,540,132,Model A,1994,2939
1  (8, 150),2016,06,PASIR RIS,219,150,Apartment,1993,3387
8
```

(b) Matching rows in `ScanResult_U2220136J.csv`.

Figure 5: Excel cross-validation: Office Script output (above) vs. engine CSV output (below). PPSM values match exactly for all tested pairs.

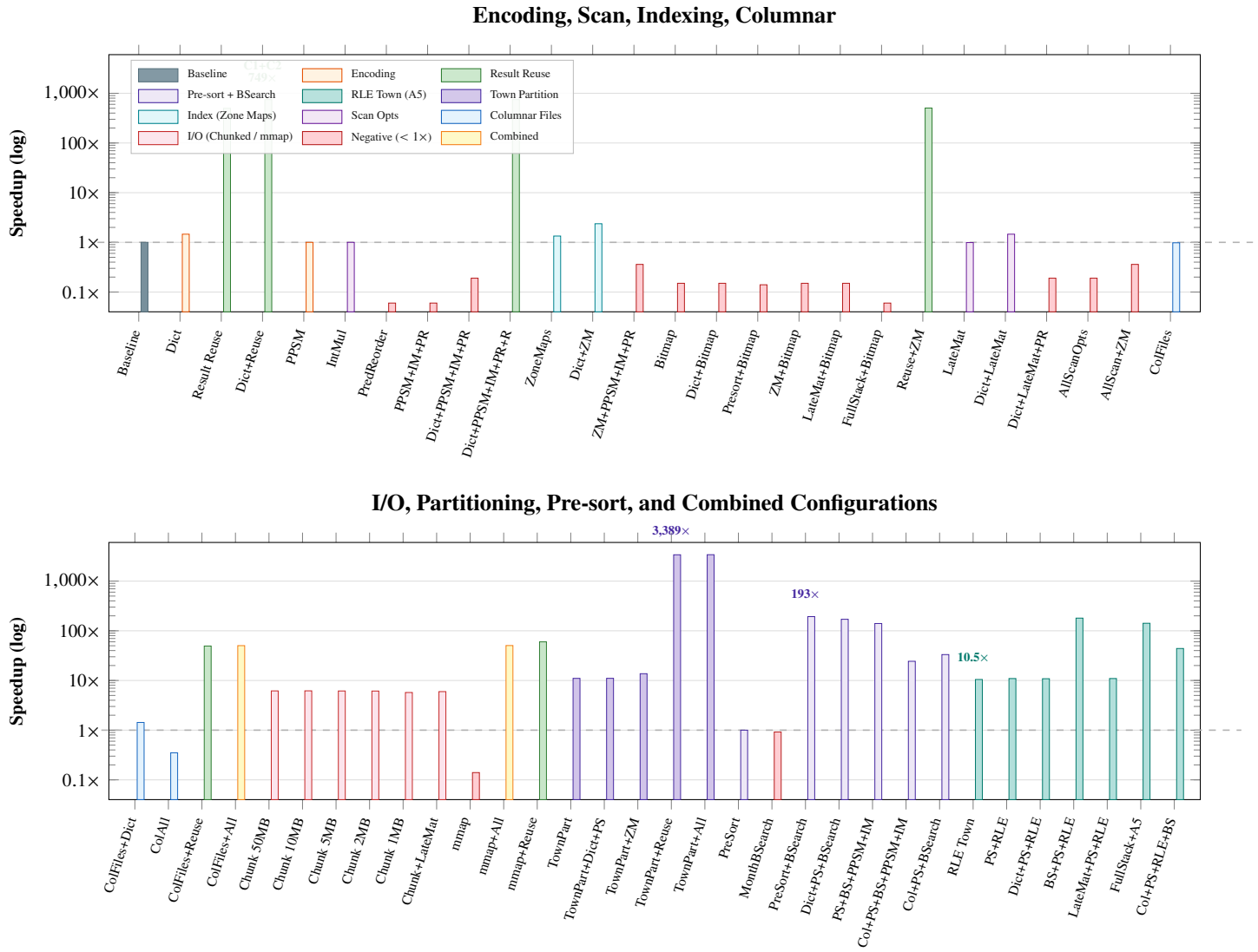


Figure 6: Speedup vs. baseline across all configurations (log scale, two rows).

```cpp
std::vector<std::vector<MinEntry>> buildCumulativeTable(
    const ColumnStore& db, uint16_t target_year,
    uint8_t start_month, const std::vector<std::string>& towns) {
  const std::size_t N = db.size();
  std::vector<std::vector<MinEntry>> per_x(9, std::vector<MinEntry>(151));
  // Pre-resolve town IDs (dict-encoded) or build hash set (string path)
  std::unordered_set<std::string> town_set;
  std::vector<uint16_t> town_ids;
  if (db.use_dict_encoding)
    for (auto& t : towns) { uint16_t id; if (db.dict_town.lookup(t,id))
        town_ids.push_back(id); }
  else town_set.insert(towns.begin(), towns.end());

  // --- Step 1: Single O(N) scan --- per-(offset, bucket) minimum ---
  for (std::size_t i = 0; i < N; ++i) {
    if (db.col_month_year[i] != target_year) continue;  // year filter
    const uint8_t month = db.col_month_month[i];
    if (month < start_month) continue;
    int offset = int(month) - int(start_month) + 1;
    if (offset < 1 || offset > 8) continue;              // month range
    if (db.use_dict_encoding) {                          // town filter
      bool match = false; uint16_t rid = db.col_town_encoded[i];
      for (auto& tid : town_ids) if (rid == tid) { match=true; break; }
      if (!match) continue;
    } else { if (town_set.find(db.col_town[i]) == town_set.end()) continue
        ; }
    const unsigned area = db.col_floor_area[i];
    if (area < 80) continue;                             // below all y
    const unsigned bucket = (area > 150) ? 150u : area;  // clamp
    const double ppsm = db.use_precomputed_ppsm
       ? db.col_price_per_sqm[i]
       : double(db.col_resale_price[i]) / double(db.col_floor_area[i]);
    MinEntry& e = per_x[offset][bucket];
    if (!e.has || ppsm < e.ppsm) { e.has=true; e.ppsm=ppsm; e.idx=i; }
  }
  // --- Step 2: X-sweep --- cumulative min across month offsets ---
  std::vector<std::vector<MinEntry>> cum_x(9, std::vector<MinEntry>(151));
  for (int area = 80; area <= 150; ++area) {
    MinEntry running;
    for (int off = 1; off <= 8; ++off) {
      if (per_x[off][area].has)
        if (!running.has || per_x[off][area].ppsm < running.ppsm) running = per_x[off][area];
      cum_x[off][area] = running;
    }
  }
  // --- Step 3: Y-sweep --- propagate min from higher to lower area ---
  for (int off = 1; off <= 8; ++off)
    for (int area = 149; area >= 80; --area) {
      const MinEntry& hi = cum_x[off][area+1]; MinEntry& lo = cum_x[off][area];
      if (hi.has && (!lo.has || hi.ppsm < lo.ppsm)) lo = hi;
    }
  return cum_x; // O(1) lookup: cum_x[x][y]
}
```

Complexity: Step 1 = $O(N)$ | Steps 2+3 = $O(1)$ | Query: $568 \times O(1)$ lookups

Figure 7: C++ implementation of `buildCumulativeTable` from `query_engine.cpp`.